DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim

Student Name: Trapti Kulshrestha

STUDENT ID: 019124833

Homework: 2

Section 0: Personal Configuration:

SID4	4833
PORT_BASE:	8333
PREFIX	s4833
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID:	1
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Air M4

Local Model: qwen3:8b

Tagged Commit Hash: fa2ffdb

GitHub Repository: *https://github.com/Psycoder0611/data260-4833/tree/hw02-final*

I continued using the same data260-4833 repository from HW1 & extended the existing codebase for this homework, as required.

Part 1 – HTML & CSS

Responsive Clinical Trial Interface:

For this part I extended my existing HW1 Clinical Trial Listings form instead of creating a new application. I updated the styling so that the page remains usable on a 375 px wide screen & added visible loading, empty, and error states.

The main responsive CSS I added is:




```
<head>
    <style>
        I am adjusting the layout specifically for 375px screens,
        because HW2 requires the page to remain usable at this width.
    */
    @media (max-width: 375px) {

        body {
            padding: 16px 12px;
        }

        h1 {
            font-size: 26px;
            line-height: 1.2;
        }

        .subtitle {
            font-size: 14px;
            margin-bottom: 20px;
        }

        .card {
            padding: 16px;
            border-radius: 10px;
            margin-bottom: 18px;
        }

        .card h2 {
            font-size: 21px;
        }

        input,
        select,
```

I also added three interface states:

```
<body>

    <!-- This section will display the records returned from FastAPI -->
    <section class="card">

        <h2>Available Clinical Trials</h2>

        <!-- I am showing this while records are loading -->
        <div
            id="loadingState"
            class="status-message"
        >
            Loading clinical trials...
        </div>

        <!-- I am showing this when no matching records are found -->
        <div
            id="emptyState"
            class="status-message hidden"
        >
            No clinical trials found.
        </div>

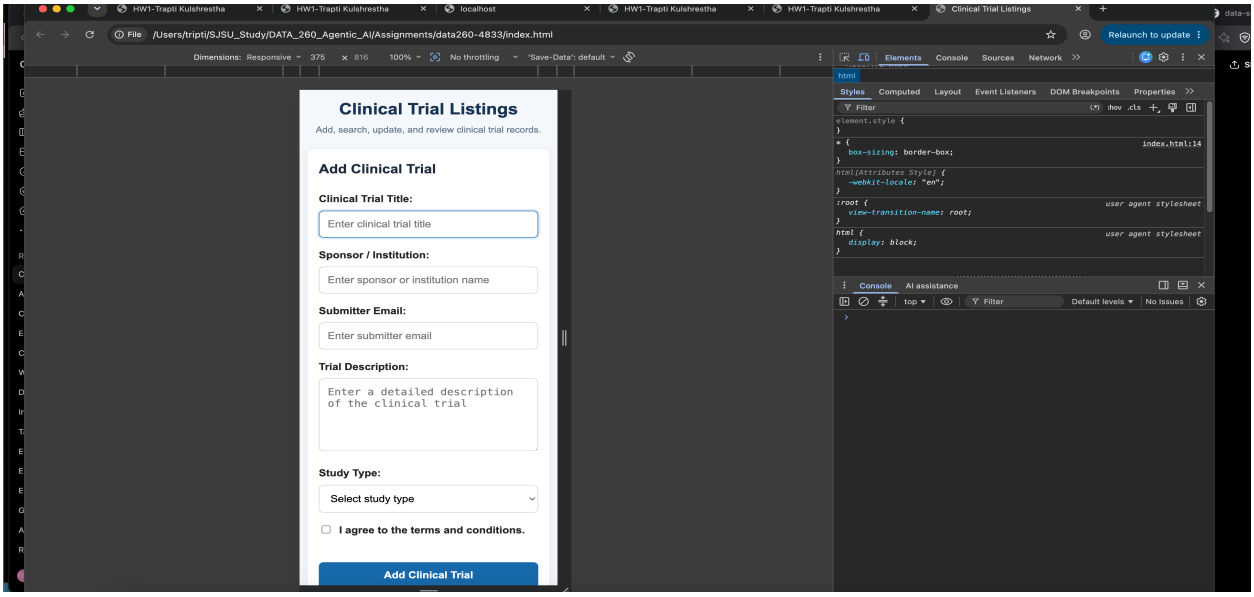
        <!-- I am showing this when the API request fails -->
        <div
            id="errorState"
            class="status-message hidden"
        >
            Unable to load clinical trials. Please try again.
        </div>

        <!-- JavaScript will insert the clinical trial records here -->
        <div id="trialList"></div>
```

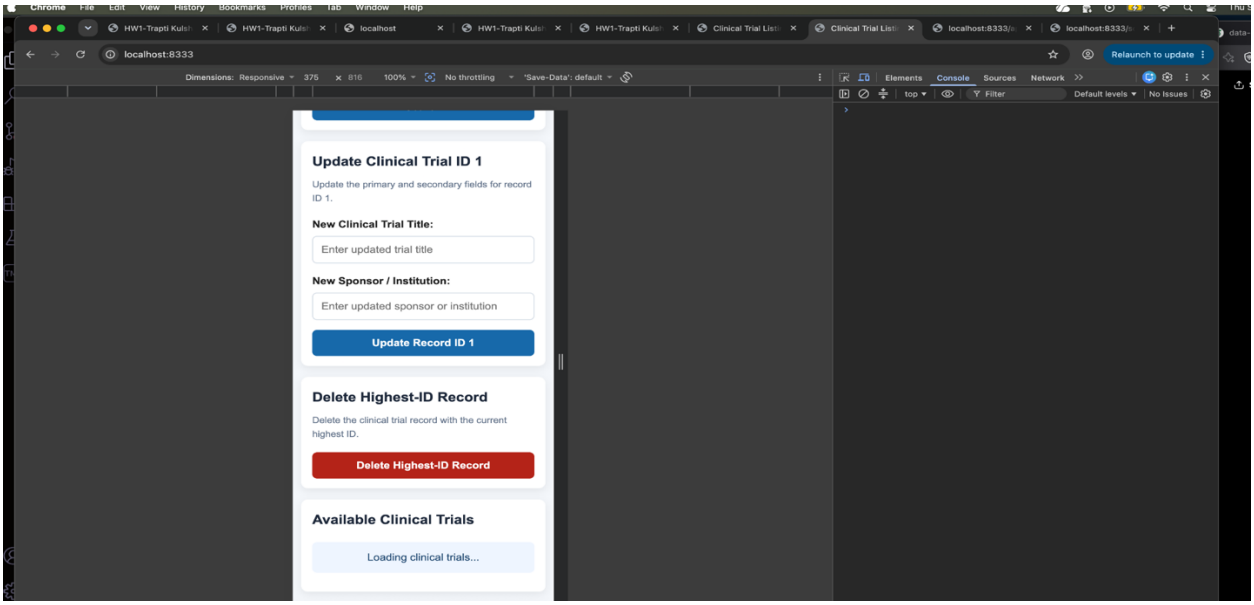
OBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

These states are controlled by JavaScript depending on the response received from the backend.

Screenshot 1 – 375 px responsive interface



Screenshot 2 – Loading state at 375 px width



The interface displays a visible loading state while the frontend waits for data from the FastAPI backend. This confirms that the page provides user feedback during API loading.

Part 2 – FastAPI

I created a FastAPI backend running on my assigned port: `PORT_BASE = 8333`

The main API model represents a clinical trial record:

```python
from fastapi import FastAPI, HTTPException, Query
from fastapi.responses import FileResponse
from pydantic import BaseModel, EmailStr
from typing import List, Optional
import uvicorn


# I am creating one FastAPI application for my Clinical Trial Listings domain.
app = FastAPI(
    title="Clinical Trial Listings API",
    version="2.0.0"
)


# ---------------------------------------------------------
# Pydantic models
# ---------------------------------------------------------

# This model represents one complete clinical trial record.
class ClinicalTrial(BaseModel):
    id: int
    trialTitle: str
    sponsor: str
    submitterEmail: str
    description: str
    category: str


# I am using a separate model for creating a new record
# because the user does not provide the ID manually.
class ClinicalTrialCreate(BaseModel):
    trialTitle: str
    sponsor: str
    submitterEmail: str
```

The application is served from: *http://localhost:8333*

The API endpoint is: http://localhost:8333/api/trials

Question 1: Add a New Record

I implemented a POST /api/trials endpoint. The new record ID is generated as one more than the current maximum ID.



```python
# ------------------------------------------------------------
# CREATE
# HW2 Part 2, Question 1
# ------------------------------------------------------------

@app.post(
    "/api/trials",
    response_model=ClinicalTrial,
    status_code=201
)
async def create_trial(
    trial_data: ClinicalTrialCreate
):
    """
    I am adding a new clinical trial record.

    The new ID is always one greater than the current
    highest ID.
    """

    # I am checking the two main domain fields manually
    # so whitespace-only values are not accepted.
    if not trial_data.trialTitle.strip():
        raise HTTPException(
            status_code=400,
            detail="Clinical trial title is required."
        )

    if not trial_data.sponsor.strip():
        raise HTTPException(
            status_code=400,
            detail="Sponsor or institution is required."
        )
```

```python
    # I am finding the current highest ID.
    highest_id = max(
        (trial.id for trial in clinical_trials),
        default=0
    )

    new_trial = ClinicalTrial(
        id=highest_id + 1,
        trialTitle=trial_data.trialTitle.strip(),
        sponsor=trial_data.sponsor.strip(),
        submitterEmail=trial_data.submitterEmail.strip(),
        description=trial_data.description.strip(),
        category=trial_data.category
    )

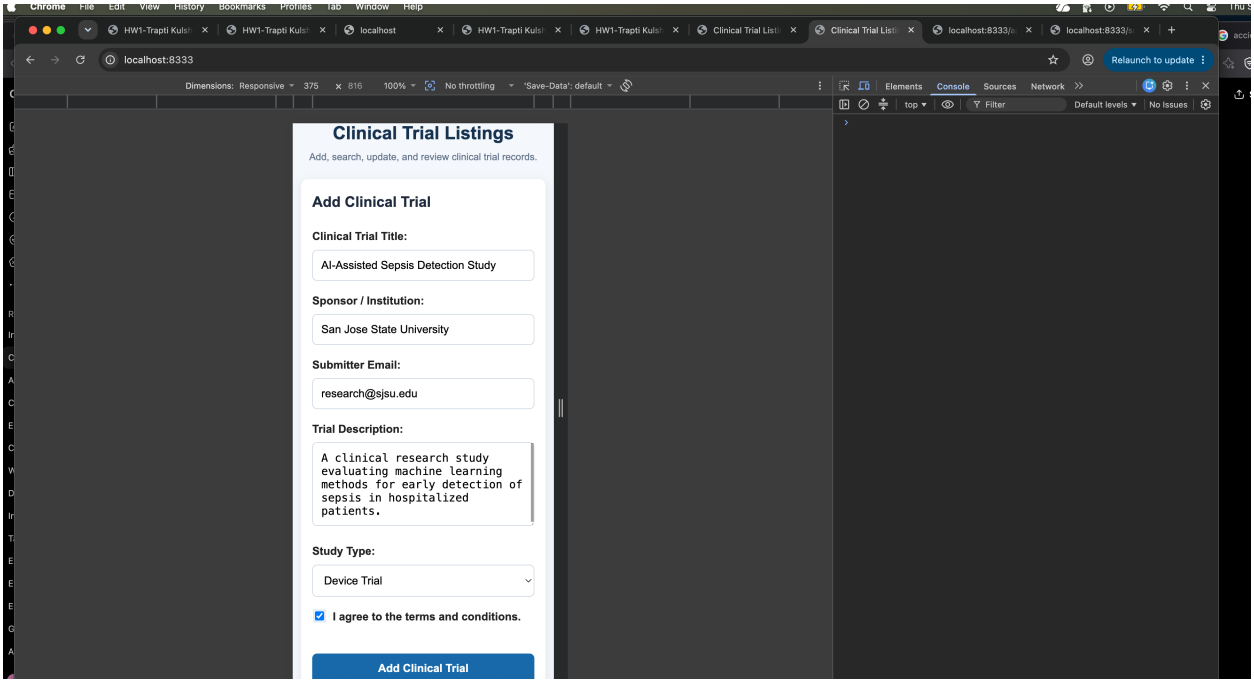
    clinical_trials.append(new_trial)

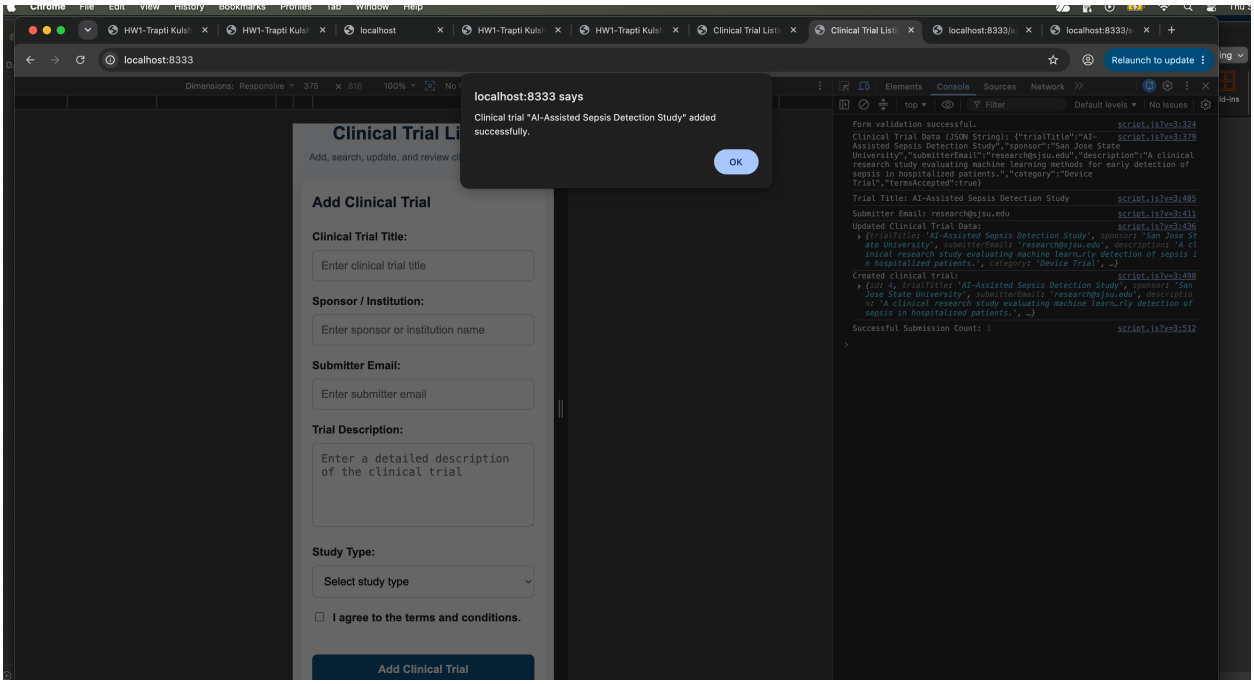
    print(f"Created clinical trial: {new_trial}")

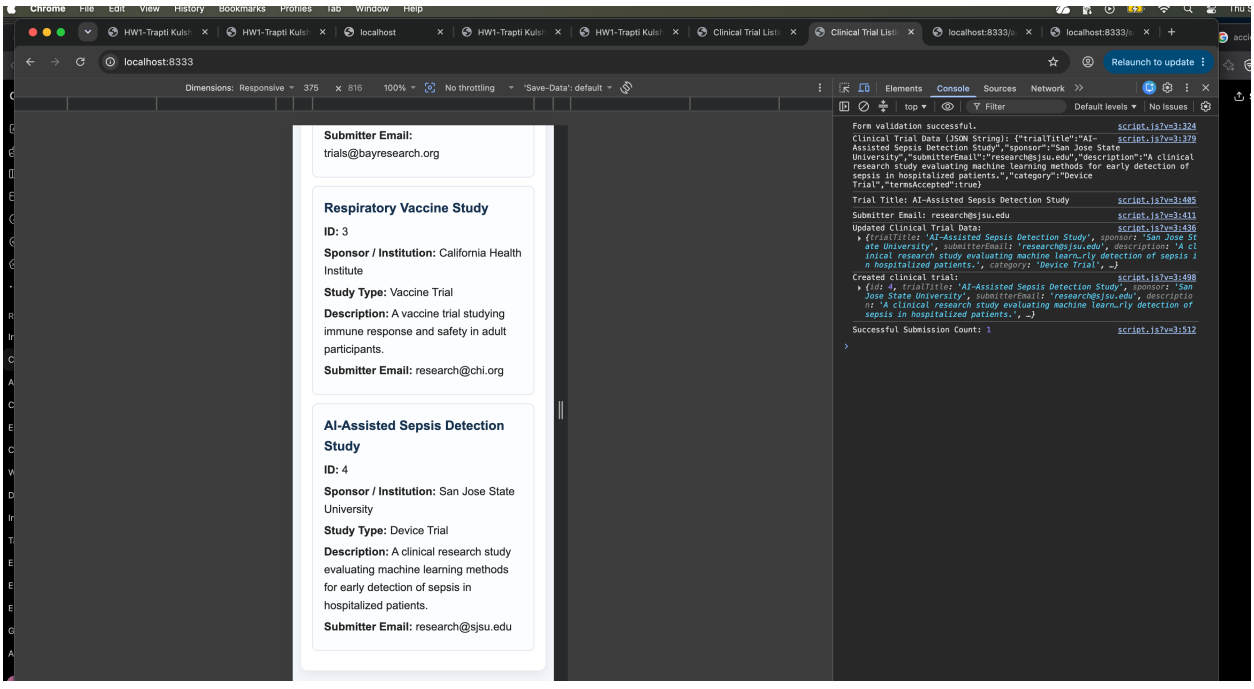
    return new_trial
```

For testing, I added:

Screenshot 3 – Added record visible in the home list

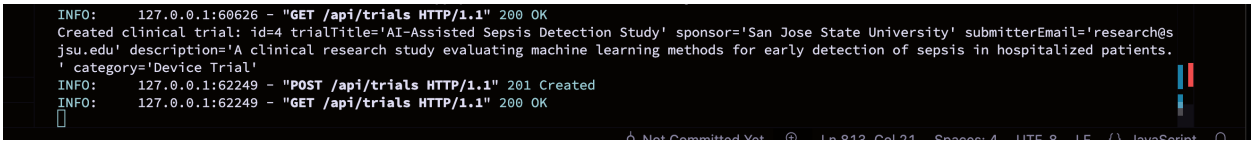






The new record was successfully created as ID 4, and FastAPI returned:

Screenshot 4 – Terminal showing 201 Created



The new record is created successfully and immediately appears in the updated home list.

Question 2: Update Record ID 1

I implemented the update endpoint using:

```python
# ------------------------------------------------------------
# UPDATE
# HW2 Part 2, Question 2
# ------------------------------------------------------------

@app.put(
    "/api/trials/{trial_id}",
    response_model=ClinicalTrial
)
async def update_trial(
    trial_id: int,
    trial_data: ClinicalTrialUpdate
):
    """
    I am updating an existing clinical trial record.

    For the HW2 screenshot, I will specifically update ID 1.
    """

    trial = next(
        (
            trial
            for trial in clinical_trials
            if trial.id == trial_id
        ),
        None
    )

    if trial is None:
        raise HTTPException(
            status_code=404,
            detail="Clinical trial not found."
        )
```

```python
    if not trial_data.trialTitle.strip():
        raise HTTPException(
            status_code=400,
            detail="Clinical trial title is required."
        )

    if not trial_data.sponsor.strip():
        raise HTTPException(
            status_code=400,
            detail="Sponsor or institution is required."
        )

    # The assignment specifically requires updating
    # the primary and secondary domain fields.
    trial.trialTitle = trial_data.trialTitle.strip()
    trial.sponsor = trial_data.sponsor.strip()

    # I am also allowing the additional HW1 fields
    # to be updated when values are provided.
    if trial_data.submitterEmail is not None:
        trial.submitterEmail = (
            trial_data.submitterEmail.strip()
        )

    if trial_data.description is not None:
        trial.description = (
            trial_data.description.strip()
        )
```

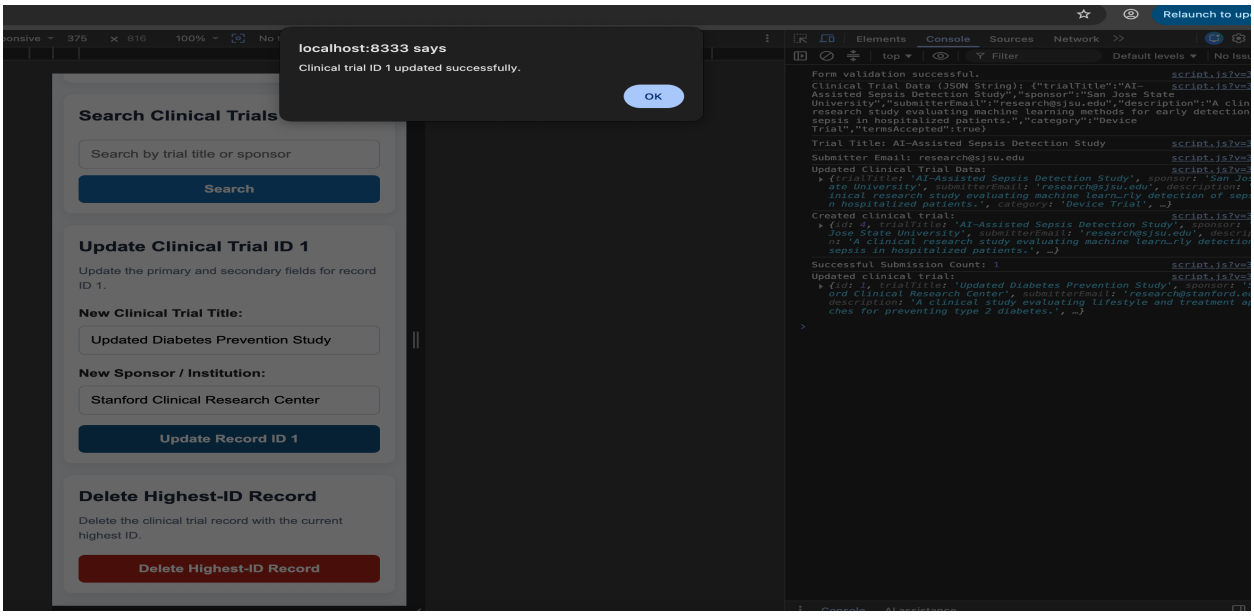
```
    if trial_data.category is not None:
        trial.category = trial_data.category

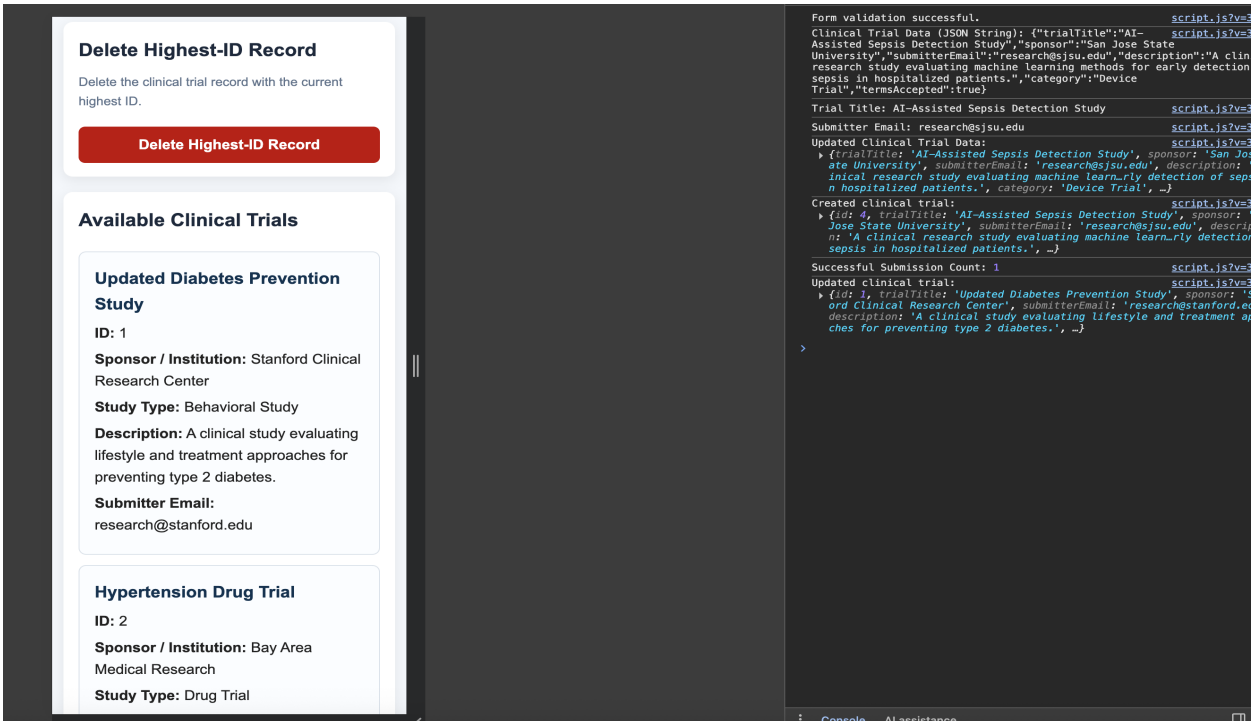
    print(f"Updated clinical trial: {trial}")

    return trial
```

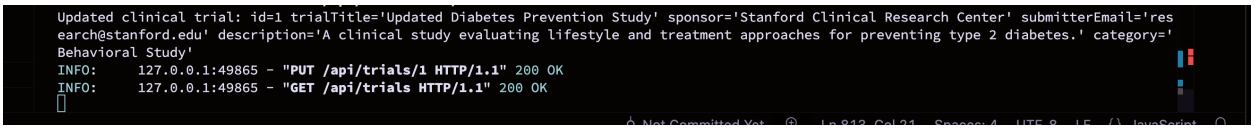
I updated ID 1 using:

Screenshot 5 – Updated ID 1 record





Screenshot 6 – PUT request in terminal



The primary and secondary fields for record ID 1 were updated successfully and the refreshed home list showed the new values.

Question 3 – Delete the Record with the Highest ID

Before deleting, the frontend retrieves all records and finds the highest ID:

```python
# ------------------------------------------------------------
# DELETE
# ------------------------------------------------------------

@app.delete(
    "/api/trials/{trial_id}",
    status_code=204
)
async def delete_trial(
    trial_id: int
):
    """
    I am deleting a clinical trial using its ID.

    The frontend will find the highest ID first and
    send that ID to this endpoint.
    """

    trial_index = next(
        (
            index
            for index, trial in enumerate(clinical_trials)
            if trial.id == trial_id
        ),
        None
    )

    if trial_index is None:
        raise HTTPException(
            status_code=404,
            detail="Clinical trial not found."
        )
```

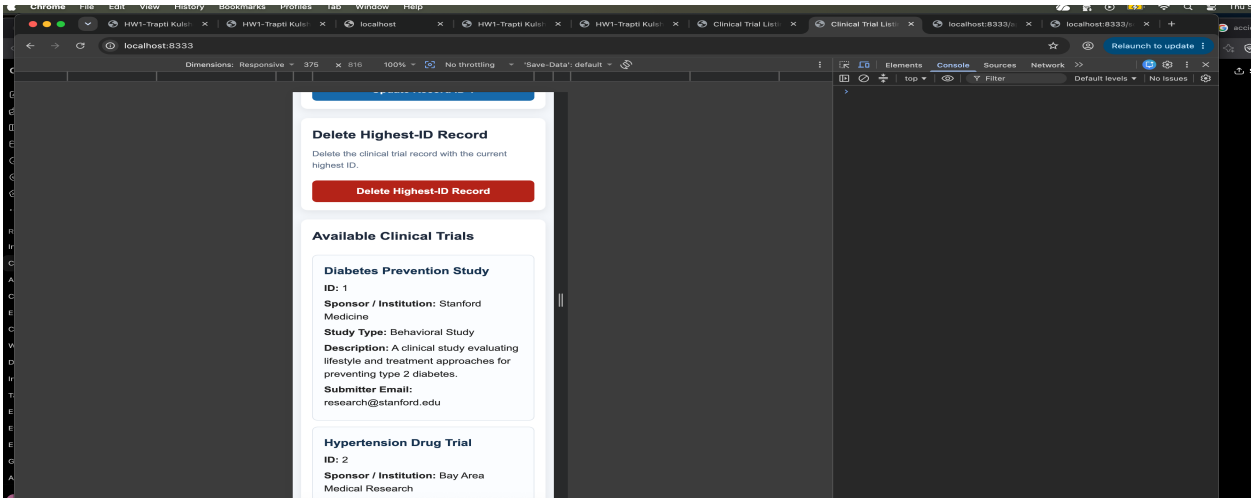
```python
    deleted_trial = clinical_trials.pop(
        trial_index
    )

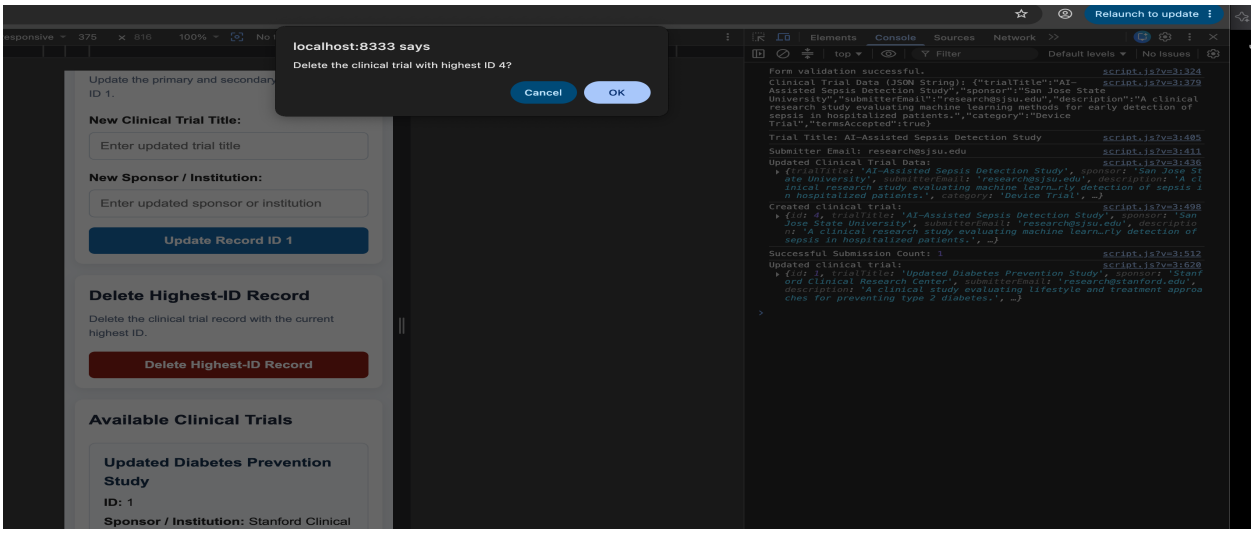
    print(
        f"Deleted clinical trial: {deleted_trial}"
    )

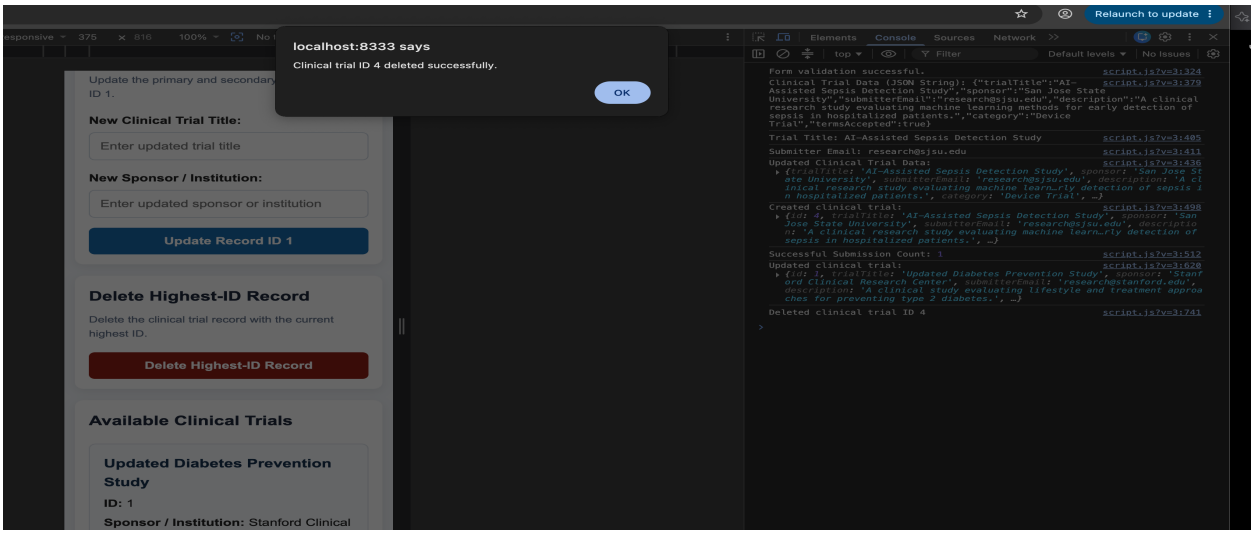
    return None
```

At the time of testing, the highest ID was 4, so ID 4 was deleted.

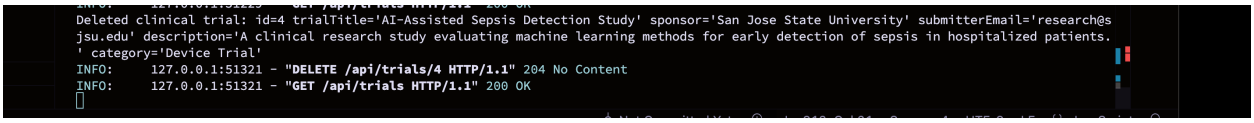
Screenshot 7 – Home list after deleting ID 4







Screenshot 8 – DELETE request output



The highest-ID record was correctly identified, deleted, and removed from the refreshed home list.

Question 4 – Search by Primary or Secondary Field

```python
# ------------------------------------------------------------
# GET records + search
# ------------------------------------------------------------

@app.get(
    "/api/trials",
    response_model=List[ClinicalTrial]
)
async def get_trials(
    q: Optional[str] = Query(
        default=None,
        description="Search by trial title or sponsor"
    )
):
    """
    I am returning all clinical trial records.

    If q is provided, I am filtering the list using
    the primary field (trialTitle) or secondary field (sponsor).
    """

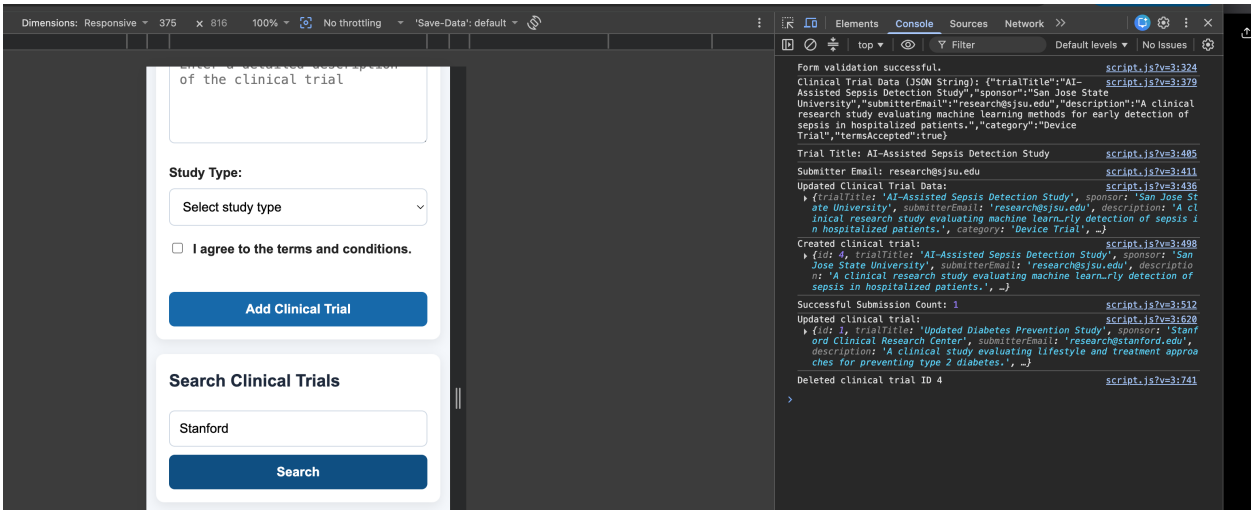
    # If there is no search value, I return the full list.
    if not q or not q.strip():
        return clinical_trials

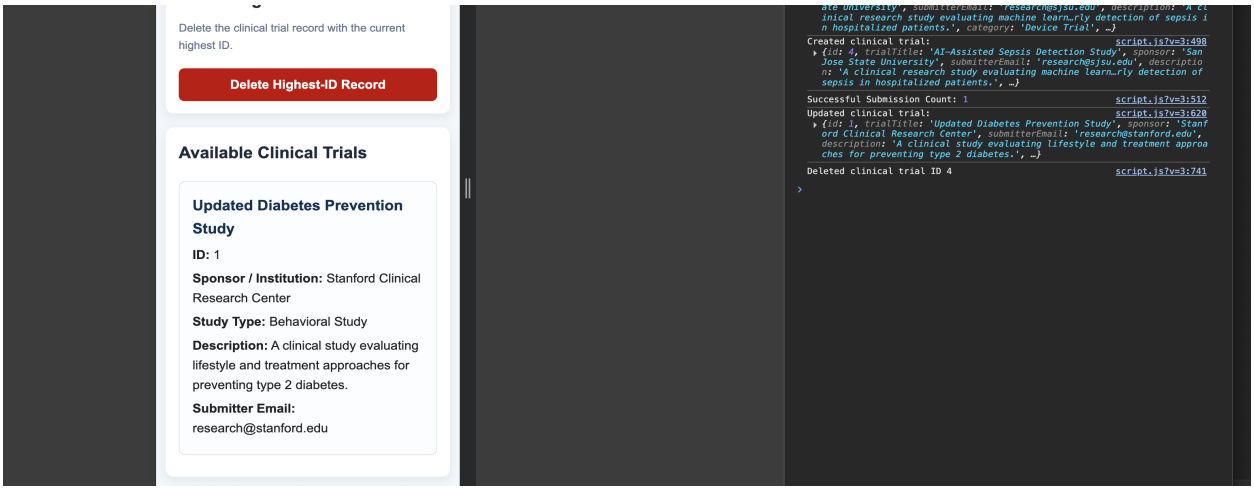
    search_value = q.strip().lower()

    # I am using case-insensitive partial matching.
    matching_trials = [
        trial
        for trial in clinical_trials
        if search_value in trial.trialTitle.lower()
        or search_value in trial.sponsor.lower()
    ]

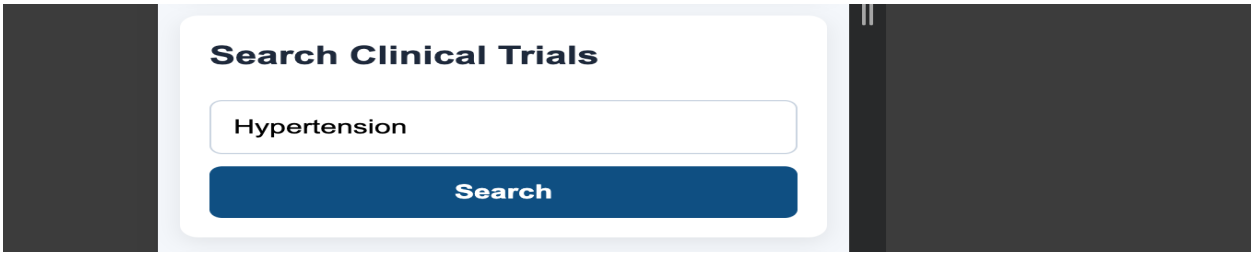
    return matching_trials
```

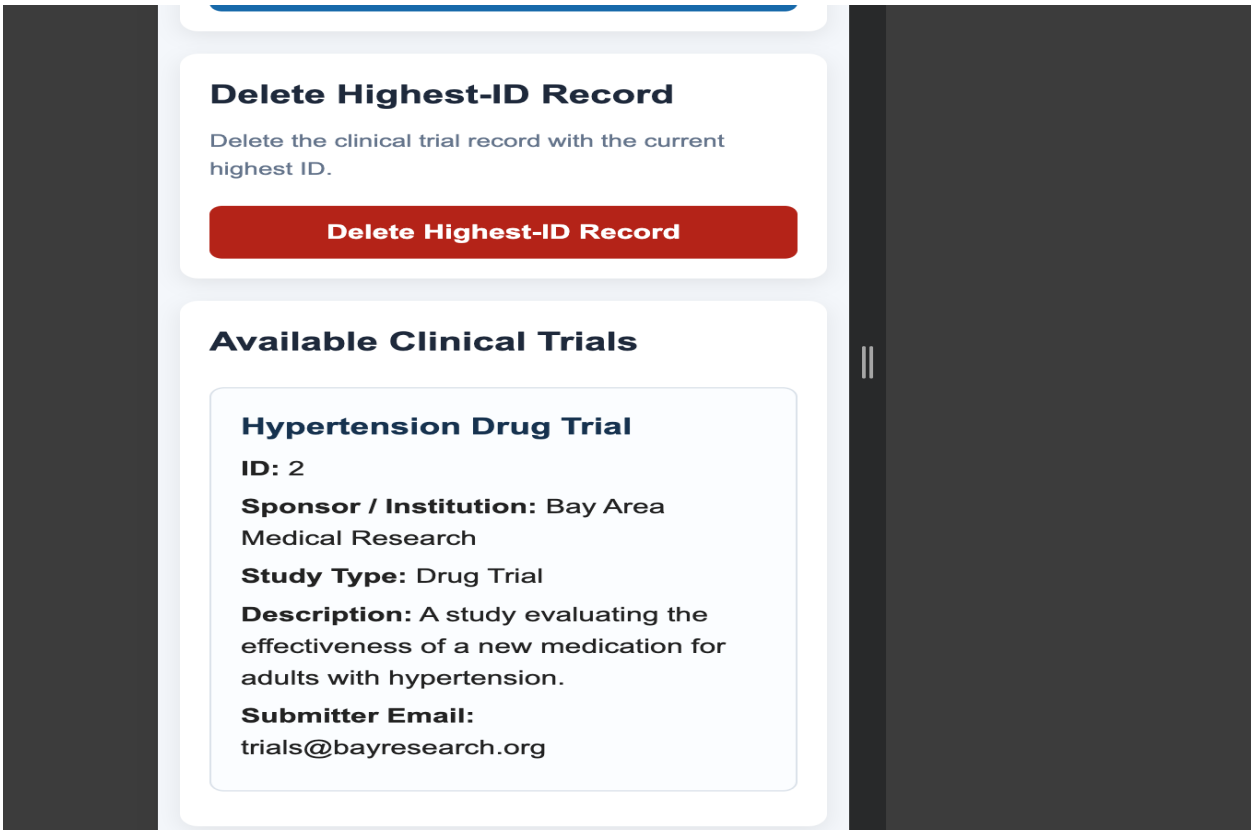
Screenshot 9 – Search result using Stanford or Hypertension

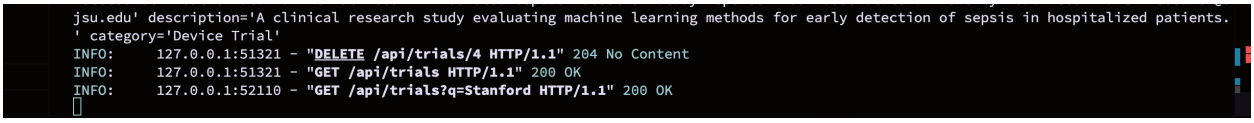




2nd Example from Primary field search:







The search correctly filters records using either the primary field or the secondary field, as required.

Part 3: Stateful Agent Graph Using LangGraph

The goal of this part was to replace the HW1 sequential Planner → Reviewer workflow with a stateful graph that can route dynamically and loop back for correction.

AgentState: The shared state acts as memory across the graph.

```python
import json

from src.model_client import ModelClient
from typing import TypedDict, Dict, Any
from langgraph.graph import StateGraph, START, END
from pydantic import BaseModel, Field, ValidationError, field_validator


# I am defining the shared state that moves through my LangGraph.
# Every node can read values from this state and return updates to it.
class AgentState(TypedDict):

    # Original input fields
    title: str
    content: str
    email: str
    strict: bool
    task: str

    # I am storing my reusable model client here
    # so Planner and Reviewer do not create ChatOllama directly.
    llm: Any

    # Planner output
    planner_proposal: Dict[str, Any]

    # Reviewer output
    reviewer_feedback: Dict[str, Any]

    # I am using this to prevent an infinite correction loop.
    turn_count: int

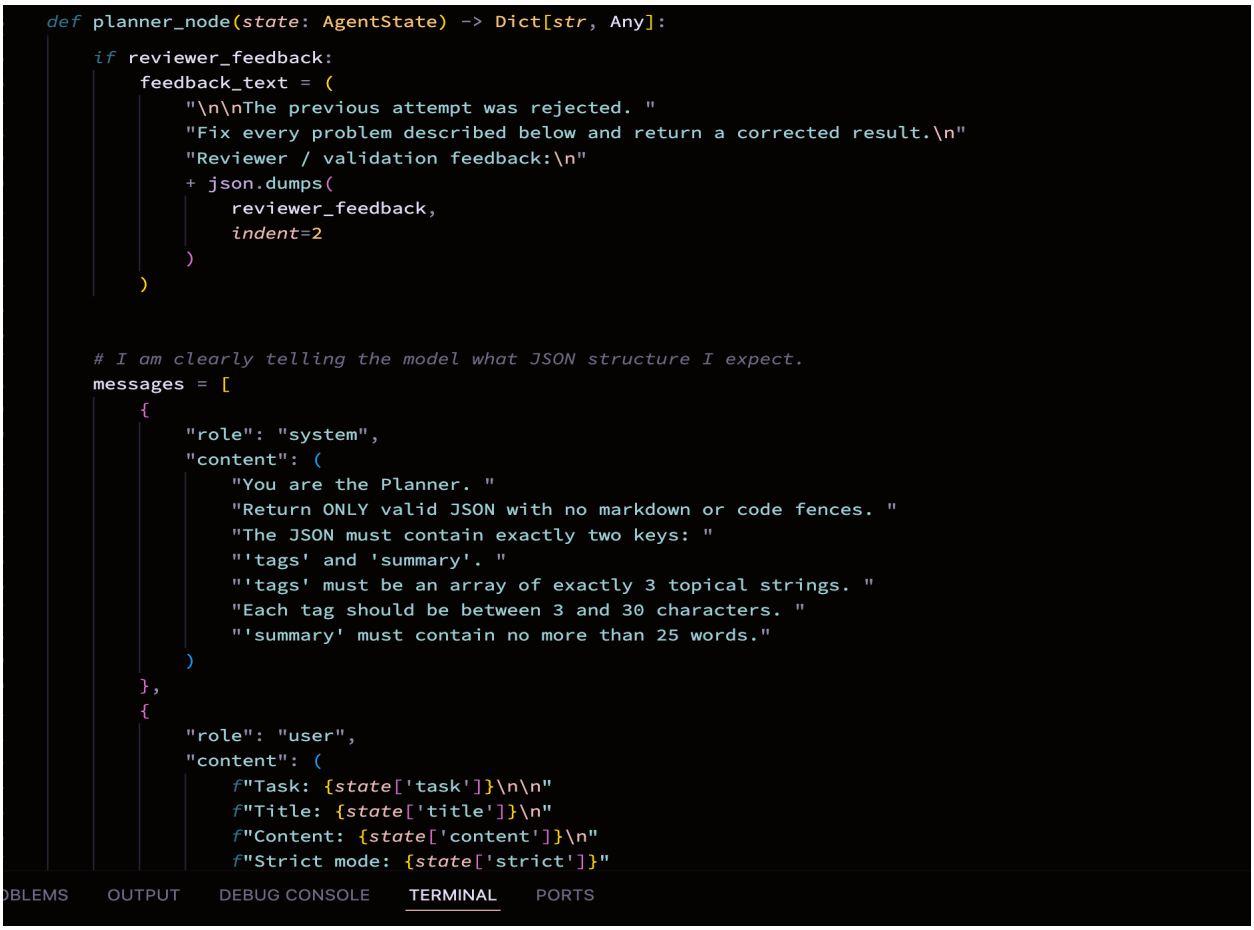
    # I will use this later for the maximum allowed graph turns.
    turn_ceiling: int
```

```python
    # I am counting Planner attempts separately so I can
    # classify first-attempt and retry outcomes in my experiments.
    planner_attempts: int


#
```

Planner Node: *The Planner generates exactly three topical tags and a short summary.*





All model calls go through: `src/model_client.py` rather than directly calling Ollama or LangChain inside the node, as required.

Reviewer Node: The Reviewer first checks the Planner output using Pydantic and then performs a quality review:

```python
def reviewer_node(state: AgentState) -> Dict[str, Any]:
    """
    I am using the Reviewer node to first validate
    the Planner output with Pydantic and then review its quality.
    """

    print("--- NODE: Reviewer ---")

    llm: ModelClient = state["llm"]

    planner_proposal = state.get(
        "planner_proposal",
        {}
    )


    # ------------------------------------------------------
    # Step 1: Pydantic schema validation
    # ------------------------------------------------------

    try:

        validated_output = PlannerOutput.model_validate(
            planner_proposal
        )

        print(
            "--- PYDANTIC: Planner output is valid ---"
        )


    except ValidationError as error:
```

If validation fails, the error is returned through reviewer_feedback.

Supervisor and Router:

The Supervisor increments the state counter:

```python
def supervisor_node(state: AgentState) -> Dict[str, Any]:
    """
    I am using the Supervisor node to update
    the shared state before deciding where to go next.
    """

    print("--- NODE: Supervisor ---")

    # I am increasing the turn counter each time
    # the Supervisor is reached.
    current_turn = state.get(
        "turn_count",
        0
    )

    new_turn = current_turn + 1

    print(
        f"Turn count: {new_turn}"
    )

    return {
        "turn_count": new_turn
    }
```

The router decides whether to send the graph to: planner -> reviewer -> END.

The graph structure is:

START -> Supervisor -> Planner -> Supervisor -> Reviewer ->

Supervisor

├── Approved → END

└── Issues → Planner

Graph Assembly:

```
def build_graph():
    """
    I am assembling my Planner, Reviewer and Supervisor
    nodes into one stateful LangGraph workflow.
    """

    # I am creating a graph that uses AgentState
    # as the shared memory between all nodes.
    workflow = StateGraph(AgentState)

    # I am registering each Python function as a graph node.
    workflow.add_node(
        "supervisor",
        supervisor_node
    )

    workflow.add_node(
        "planner",
        planner_node
    )

    workflow.add_node(
        "reviewer",
        reviewer_node
    )


    # The Supervisor is the first node that runs.
    workflow.add_edge(
        START,
        "supervisor"
    )
```

The graph is run using .stream():

```python
# HW2 specifically asks me to use .stream()
# so I can observe the output from each graph step.
for event in graph.stream(initial_state):

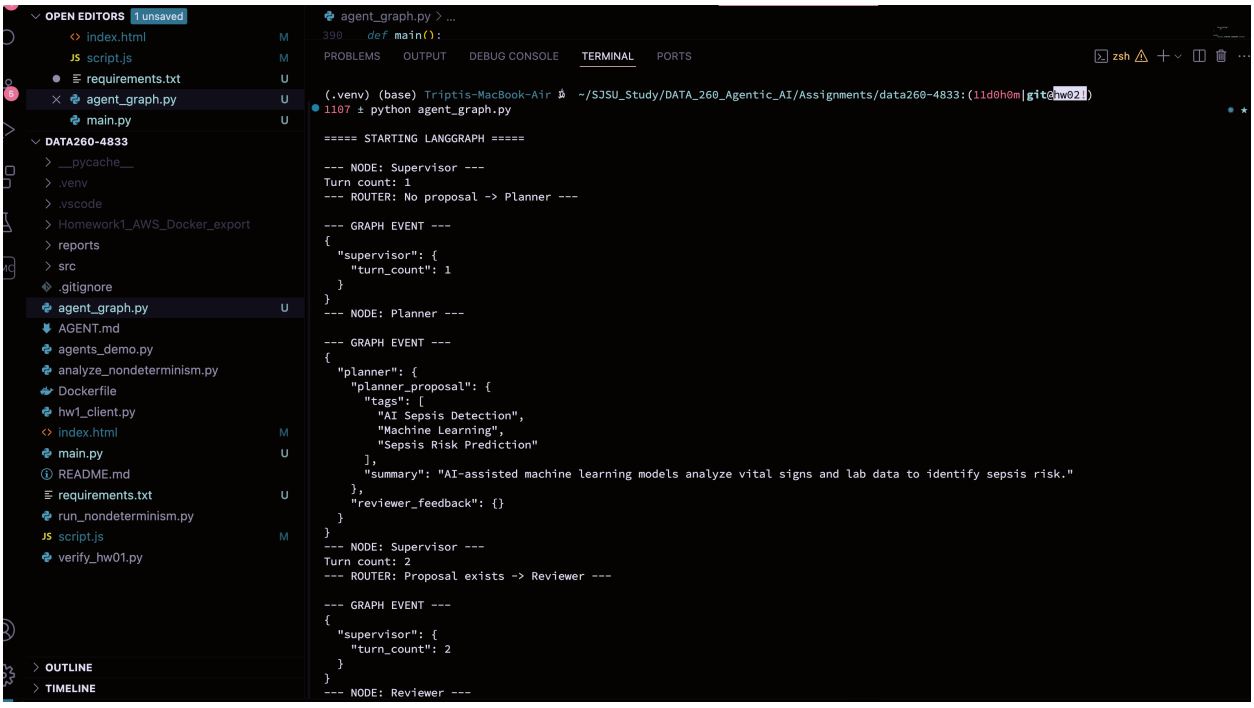
    print(
        "\n--- GRAPH EVENT ---"
    )

    print(
        json.dumps(
            event,
            indent=2,
            default=str
        )
    )

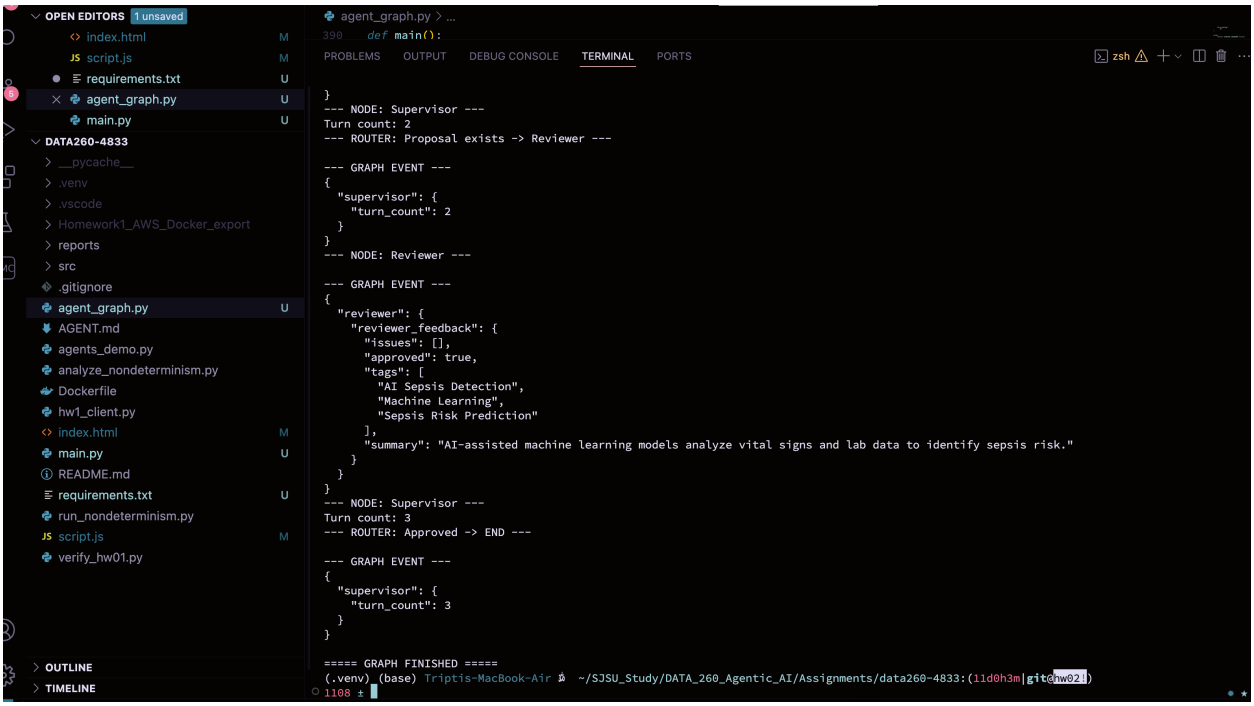

print(
    "\n===== GRAPH FINISHED ====="
)
```

This allows each graph step to be observed in the terminal, which is specifically the requirement.

Screenshot 10a – LangGraph execution begins with Supervisor, Planner, and Reviewer nodes



Screenshot 10b – Reviewer approves the proposal and the Supervisor routes the graph to END



Screenshot 11 – Supervisor routes the workflow back to the Planner after Reviewer issues are detected



The LangGraph implementation successfully converts the rigid HW1 waterfall into a stateful workflow with conditional routing and self-correction.

Part 4 – Output Schema and Loop Safety

Pydantic Output Schema-

I created a Pydantic model to enforce the required output, I then validate each tag:

```python
# ------------------------------------------------------------
# Planner output schema
# ------------------------------------------------------------

class PlannerOutput(BaseModel):
    """
    I am using Pydantic to validate the Planner output
    before the graph accepts it.
    """

    # I require exactly 3 tags.
    # Each tag must contain between 3 and 30 characters.
    tags: list[
        str
    ] = Field(
        min_length=3,
        max_length=3
    )

    summary: str


    # I am checking the length of every individual tag.
    @field_validator("tags")
    @classmethod
    def validate_tags(cls, tags):

        for tag in tags:

            if len(tag) < 3 or len(tag) > 30:

                raise ValueError(
                    "Each tag must contain between 3 and 30 characters."
                )
```

The summary is limited to 25 words:

```python


    # HW2 requires the summary to contain no more than 25 words.
    @field_validator("summary")
    @classmethod
    def validate_summary(cls, summary):

        word_count = len(
            summary.split()
        )

        if word_count > 25:

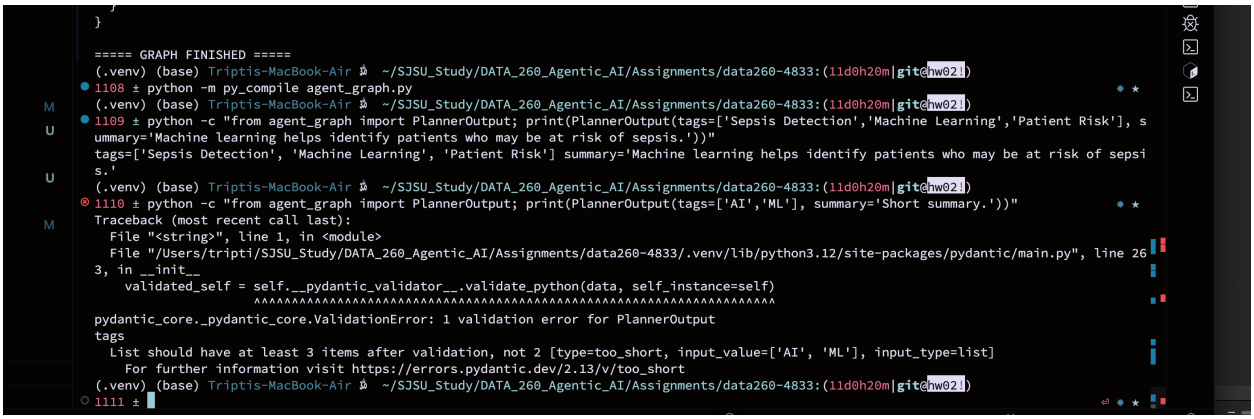
            raise ValueError(
                "Summary must contain no more than 25 words."
            )

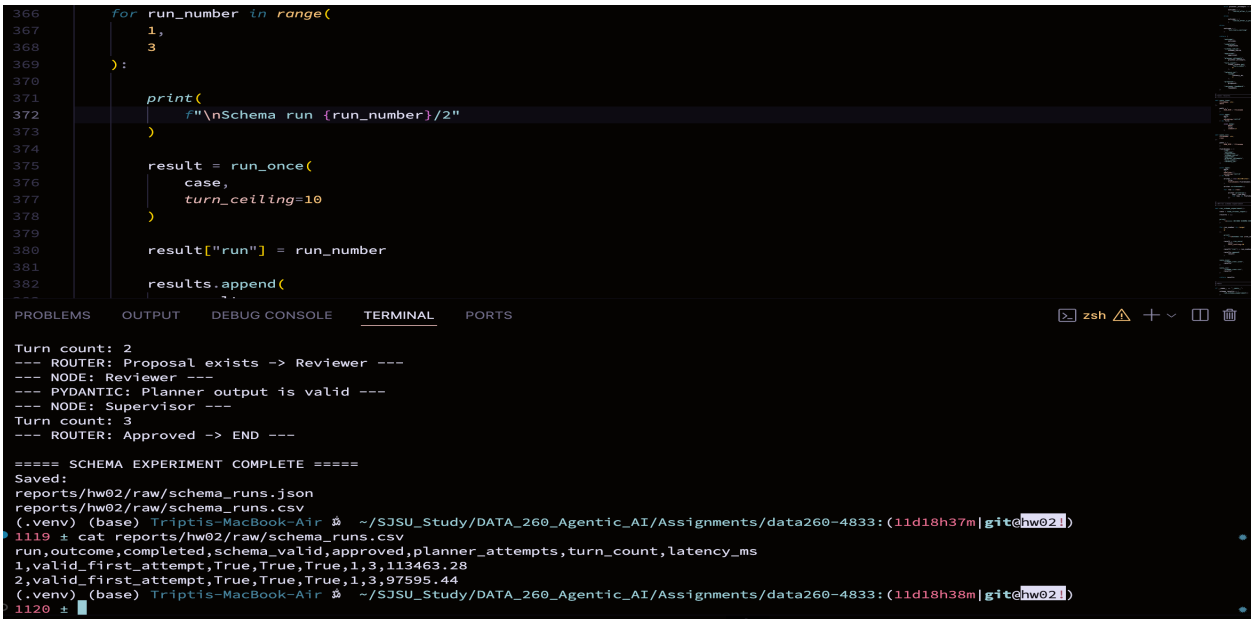
        return summary
```

Screenshot 12 – Valid schema test:

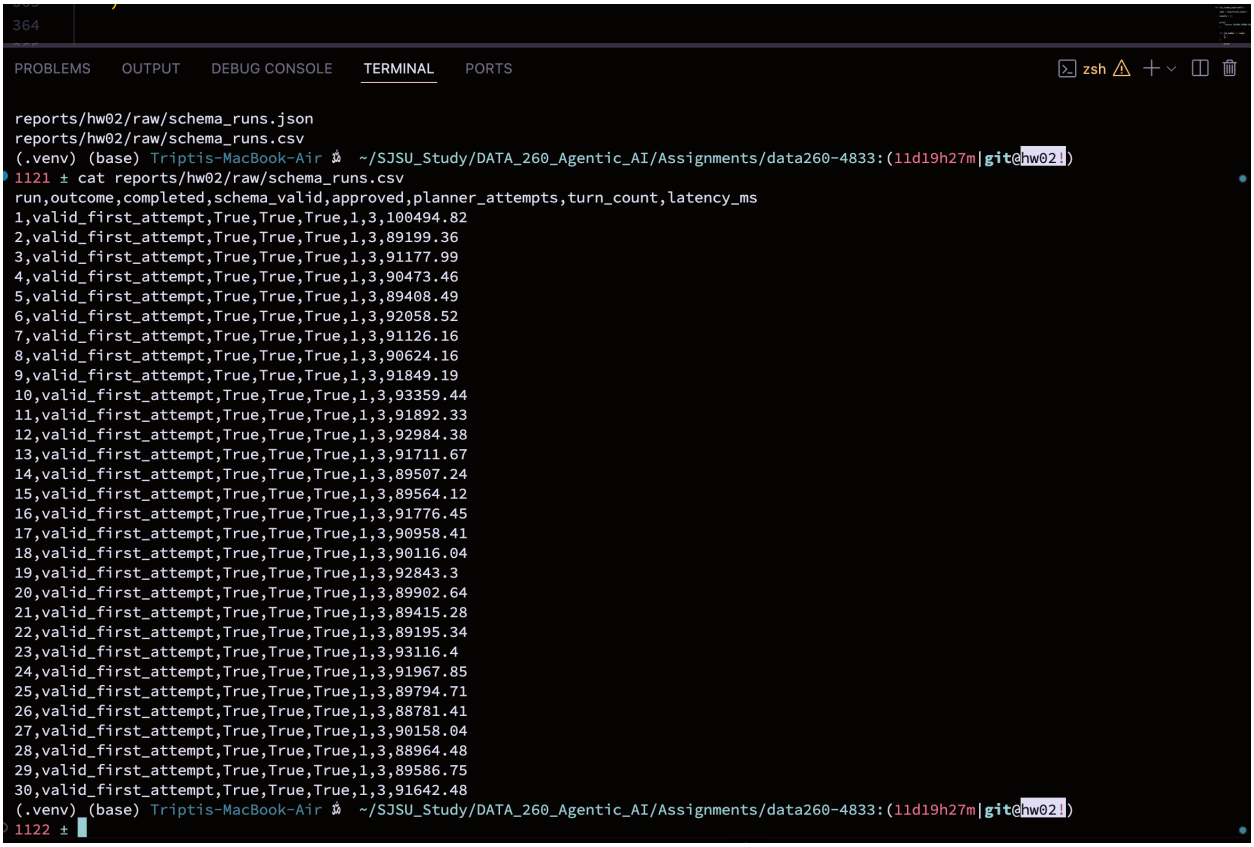


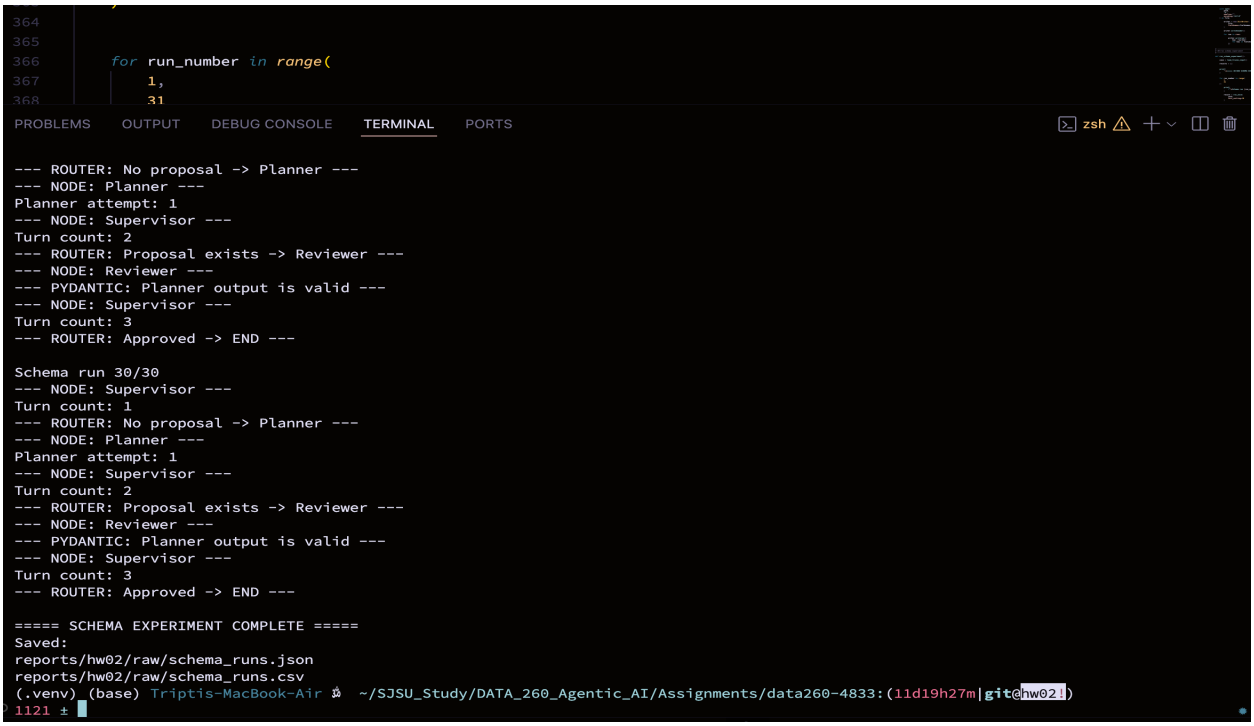
Screenshot 13 – Invalid schema test:


}
===== GRAPH FINISHED =====
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(11d0h20m|gitehw02)
1108 ± python -m py_compile agent_graph.py
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(11d0h20m|gitehw02)
1109 ± python -c "from agent_graph import PlannerOutput; print(PlannerOutput(tags=['Sepsis Detection','Machine Learning','Patient Risk'], summary='Machine learning helps identify patients who may be at risk of sepsis.'))"
tags=['Sepsis Detection', 'Machine Learning', 'Patient Risk'] summary='Machine learning helps identify patients who may be at risk of sepsis.'
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(11d0h20m|gitehw02)
1110 ± python -c "from agent_graph import PlannerOutput; print(PlannerOutput(tags=['AI','ML'], summary='Short summary.'))"
Traceback (most recent call last):
File "<string>", line 1, in <module>
File "/Users/tripti/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833/.venv/lib/python3.12/site-packages/pydantic/main.py", line 263, in __init__
validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
^^^

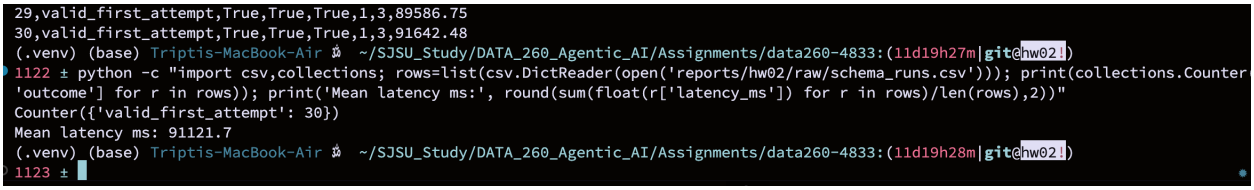


Screenshot 14 – 30/30 experiment complete





Screenshot 15 – Statistical Summary



For the fixed normal input, qwen3:8b was highly consistent and did not require correction retries.

Turn Ceiling Comparison:

I compared two turn ceilings using the same frozen input and model settings:

- Ceiling 2: 20 runs
- Ceiling 10: 20 runs

Results:

Ceiling	Completed	Completion Rate	Mean Latency
2	20/20	100%	94,650.37 ms
10	20/20	100%	89,413.78 ms

Implementation note: I apply the loop-safety ceiling to Planner attempts while retaining turn_count for graph tracing, because a raw Supervisor-visit ceiling of 2 would terminate before the Reviewer can evaluate the first proposal.



```
# Turn ceiling comparison
# 20 runs with ceiling 2 and 20 runs with ceiling 10
# ------------------------------------------------------------

def run_ceiling_comparison():

    case = load_frozen_input()

    all_results = {}


    for ceiling in [2, 10]:

        print(
            f"\n===== CEILING {ceiling} EXPERIMENT ====="
        )

        results = []


        for run_number in range(
            1,
            21
        ):

            print(
                f"\nCeiling {ceiling}: run {run_number}/20"
            )

            result = run_once(
                case,
                turn_ceiling=ceiling
            )

            result["run"] = run_number
            result["turn_ceiling"] = ceiling

            results.append(
```



```
# ------------------------------------------------------------
# Main
# ------------------------------------------------------------

if __name__ == "__main__":

    ceiling_results = (
        run_ceiling_comparison()
    )


    print(
        "\n===== CEILING COMPARISON COMPLETE ====="
    )


    print(
        "\n===== SCHEMA EXPERIMENT COMPLETE ====="
    )

    print(
        "Saved:"
    )

    print(
        "reports/hw02/raw/schema_runs.json"
    )

    print(
```

```
ROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    zsh

122 ± python -c "import csv,collections; rows=list(csv.DictReader(open('reports/hw02/raw/schema_runs.csv'))); print(collections.Counter(r[
outcome'] for r in rows)); print('Mean latency ms:', round(sum(float(r['latency_ms']) for r in rows)/len(rows),2))"
ounter({'valid_first_attempt': 30})
ean latency ms: 91121.7
.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(11d19h34m|git@hw02!)
123 ±
```

```
===== CEILING 2 EXPERIMENT =====

Ceiling 2: run 1/20
--- NODE: Supervisor ---
Turn count: 1
--- ROUTER: No proposal -> Planner ---
--- NODE: Planner ---
Planner attempt: 1
--- NODE: Supervisor ---
Turn count: 2
--- ROUTER: Proposal exists -> Reviewer ---
--- NODE: Reviewer ---
--- PYDANTIC: Planner output is valid ---
--- NODE: Supervisor ---
Turn count: 3
--- ROUTER: Approved -> END ---

Ceiling 2: run 2/20
--- NODE: Supervisor ---
Turn count: 1
--- ROUTER: No proposal -> Planner ---
--- NODE: Planner ---
Planner attempt: 1
--- NODE: Supervisor ---
Turn count: 2
```

Screenshot 16 – Ceiling comparison output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                                    zsh

--- NODE: Planner ---
Planner attempt: 1
--- NODE: Supervisor ---
Turn count: 2
--- ROUTER: Proposal exists -> Reviewer ---
--- NODE: Reviewer ---
--- PYDANTIC: Planner output is valid ---
--- NODE: Supervisor ---
Turn count: 3
--- ROUTER: Approved -> END ---

Ceiling 10 completion rate: 100.0%
Ceiling 10 mean latency: 89413.78 ms

===== CEILING COMPARISON COMPLETE =====

===== SCHEMA EXPERIMENT COMPLETE =====
Saved:
reports/hw02/raw/schema_runs.json
reports/hw02/raw/schema_runs.csv
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(11d20h38m|git@hw02!)
1125 ±
Ln 517, Col 18  Spaces: 4  UTF-8  LF  {} Python  3.12.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                                    zsh

              ^^^
SyntaxError: invalid syntax
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(11d22h22m|git@hw02!)
1126 ± python - <<'PY'
import csv

for c in [2, 10]:
    path = f"reports/hw02/raw/ceiling_{c}_runs.csv"
    rows = list(csv.DictReader(open(path)))
    done = sum(r["completed"] == "True" for r in rows)
    mean = sum(float(r["latency_ms"]) for r in rows) / len(rows)

    print(
        f"Ceiling {c}: completion={done}/{len(rows)} "
        f"({done/len(rows)*100:.1f}%), "
        f"mean latency={mean:.2f} ms"
    )
PY
Ceiling 2: completion=20/20 (100.0%), mean latency=94650.37 ms
Ceiling 10: completion=20/20 (100.0%), mean latency=89413.78 ms
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(11d22h23m|git@hw02!)
1127 ±
Ln 517, Col 18  Spaces: 4  UTF-8  LF  {} Python  3.12.9 (.ven
```

Deployment Choice:

I would choose the smaller ceiling for deployment.

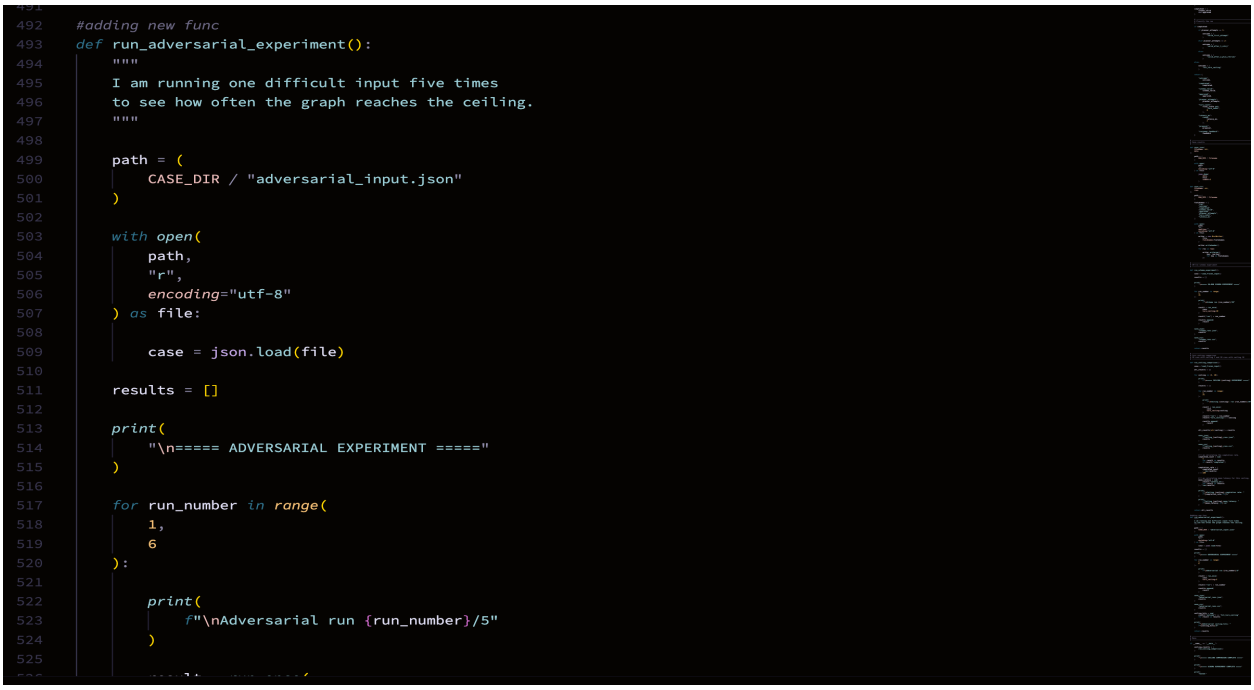
Both settings achieved the same 100% completion rate, so increasing the ceiling from 2 to 10 did not improve completion for this input. Although ceiling 10 had a slightly lower measured mean latency in this experiment, the difference appears to be normal runtime variation because the successful normal runs completed without needing multiple correction attempts.

A lower ceiling gives the workflow a tighter safety bound and reduces the risk of excessive repeated model calls if the Planner and Reviewer enter a correction loop.

Adversarial Input:

I created a difficult input that encouraged very short tags such as AI, ML, X. These violate the Pydantic rule requiring each tag to contain between 3 and 30 characters.

The observed flow was: Planner -> Reviewer -> Pydantic validation failed -> Supervisor -> Planner-attempt ceiling reached-> END



```
#adding new func
def run_adversarial_experiment():
    """
    I am running one difficult input five times
    to see how often the graph reaches the ceiling.
    """

    path = (
        CASE_DIR / "adversarial_input.json"
    )

    with open(
        path,
        "r",
        encoding="utf-8"
    ) as file:

        case = json.load(file)

    results = []

    print(
        "\n===== ADVERSARIAL EXPERIMENT ====="
    )

    for run_number in range(
        1,
        6
    ):

        print(
            f"\nAdversarial run {run_number}/5"
        )
```

```python
    except ValidationError as error:

        # I am storing the real Pydantic validation error
        # so the Planner can use it during the next retry.
        # I am keeping the validation feedback concise so the
    # Planner receives the actual problem without a very long error dump.
        validation_message = "; ".join(
            item["msg"]
            for item in error.errors()
        )

        print(
            "--- PYDANTIC: Validation failed ---"
        )

        print(validation_message)

        return {
            "reviewer_feedback": {
                "issues": [
                    validation_message
```

```python
def run_adversarial_experiment():

    for run_number in range(
        1,
        6
    ):

        print(
            f"\nAdversarial run {run_number}/5"
        )

        result = run_once(
            case,
            turn_ceiling=1
        )

        result["run"] = run_number

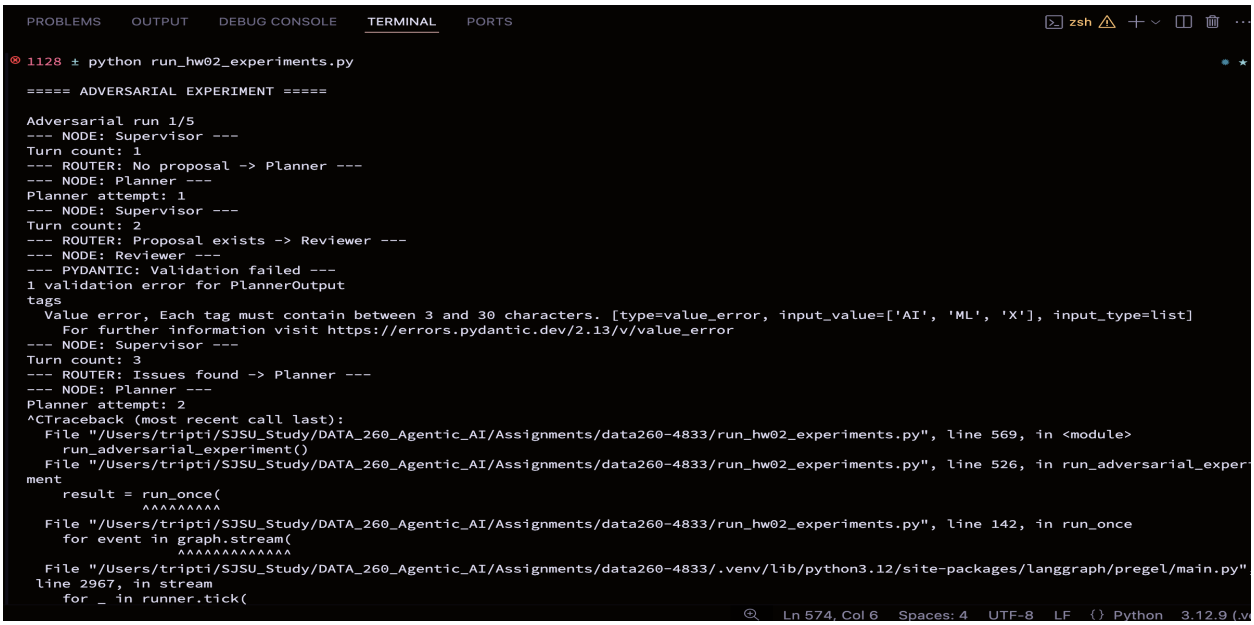
        results.append(
            result
        )

    save_json(
        "adversarial_runs.json",
        results
    )
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  zsh

          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/tripti/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833/.venv/lib/python3.12/site-packages/httpcore/_sync/http1
```

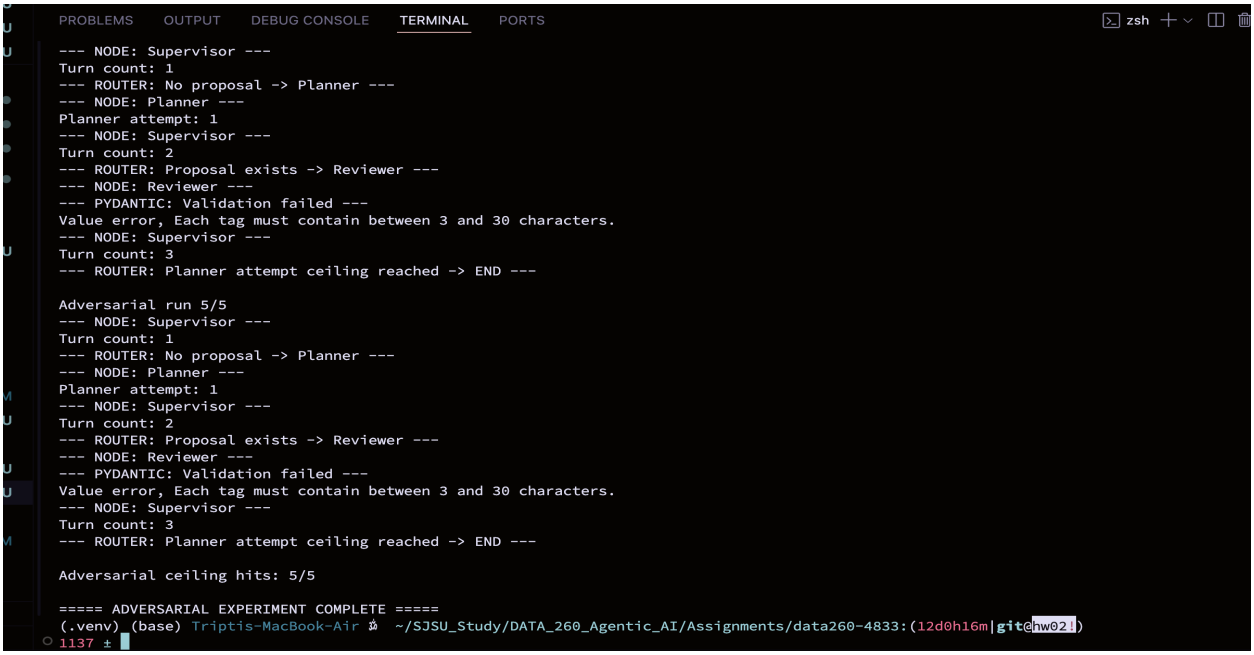
To test the failure behavior consistently, I created a separate adversarial experiment function that loads one fixed difficult input and runs it 5 times. For each run, I record the final outcome & then count how many runs reach the configured safety ceiling. The raw results are also saved to both CSV and JSON files for verification.

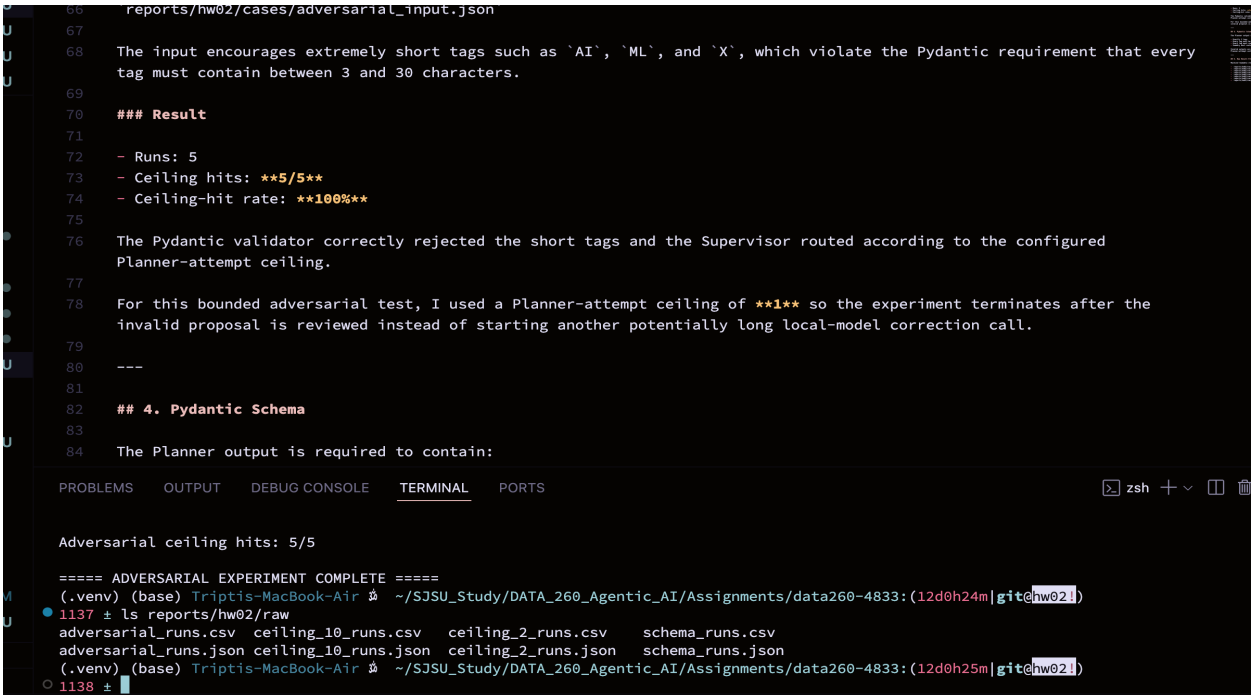


Results:

Metric	Result
Runs	5
Ceiling Hits	5
Ceiling-hit rate	100%

Screenshot 17 – Adversarial 5/5 output





Why this input causes trouble: The input intentionally pushes the Planner toward tags that are too short to satisfy the schema. This creates a conflict between the adversarial request and the system constraints.

Proposed Fix: One improvement would be to enforce structured generation more strongly before validation, for example by using a schema-aware structured-output mechanism or by adding a deterministic pre-check before sending a correction request back to the model.

During testing, a second Planner retry occasionally stalled locally with qwen3:8b. For the bounded adversarial experiment, I used a Planner-attempt ceiling of 1 so that the safety behavior could be tested reliably without allowing an unbounded local model call.

Takeaway: The validator consistently identified the malformed tags and the graph terminated safely instead of looping indefinitely. The assignment asks for an adversarial input to be run five times and for the observed ceiling rate to be reported; my case reached the ceiling in all 5 runs.

Verification:

I wrote a smoke-test script: `verify_hw02.py`

The script objectively checks:

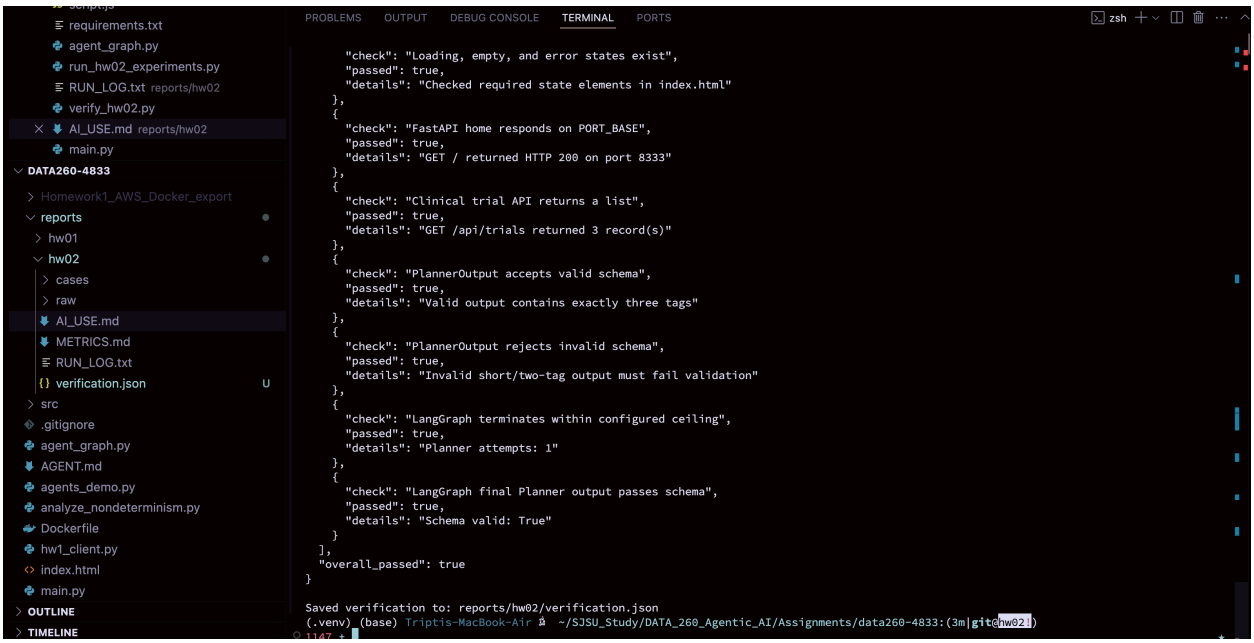
- 375 px responsive CSS
- loading / empty / error states
- FastAPI home response on port 8333
- /api/trials response
- valid Pydantic schema
- invalid schema rejection
- LangGraph termination within the configured ceiling
- final Planner output schema validity

The final result was: `"overall_passed": true`

Screenshot 18 – verification.json / overall_passed true







The machine-readable verification result is stored at: reports/hw02/verification.json

This file includes the homework number, SID4, commit hash, model/configuration, SEED, VERIFY_SEED, checks run, and pass/fail results, matching the repository requirement.

AI Use:

1. What did I use AI for, and what did I do myself?

I used ChatGPT & Claude to understand tools & terminology, review code structure, explain FastAPI & LangGraph concepts & troubleshoot errors. I personally created & modified the

files, ran all commands locally, tested the application & executed & verified all experiments.

2. What was one AI suggestion that was unsuitable, or something I verified independently?

The initial adversarial experiment used a Planner-attempt ceiling of 2, but the second Planner retry repeatedly stalled with my local qwen3:8b model. I also independently verified that Pydantic correctly rejected short tags such as AI, ML, and X.

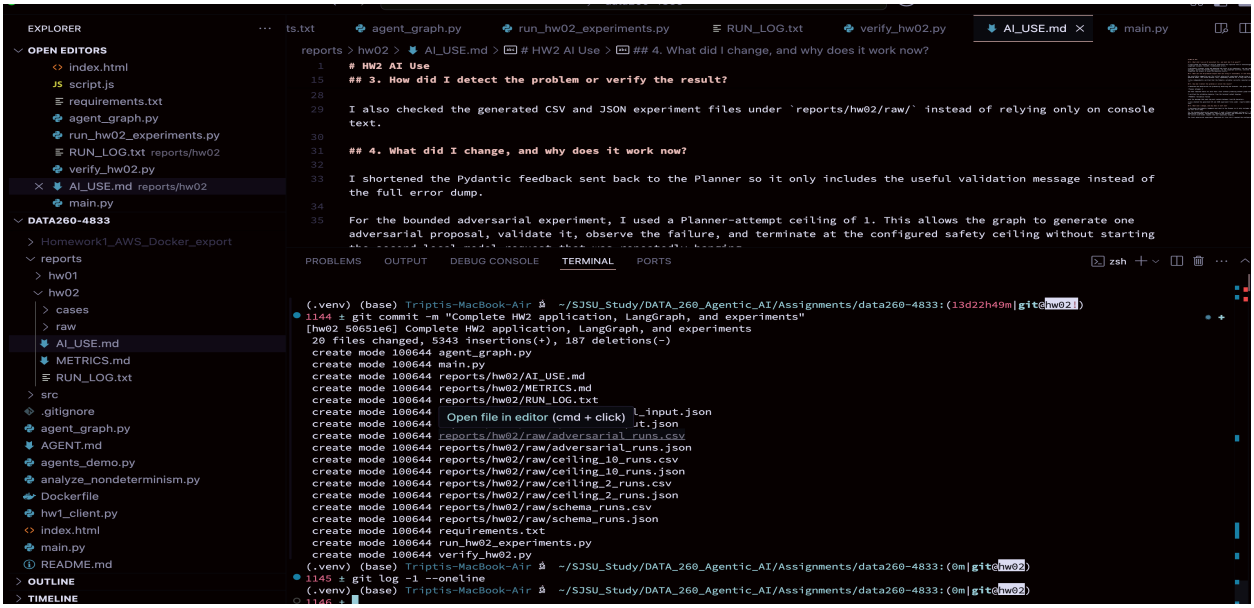
3. How did I detect or verify the issue?

I observed the terminal staying at Planner attempt: 2 for over an hour without another graph event. I verified the validation behavior through *PYDANTIC: Validation failed messages* & by checking the saved CSV/JSON experiment outputs.

4. What did I change, and why does it work now?

I shortened the validation feedback sent back to the Planner and used a bounded adversarial test with a Planner-attempt ceiling of 1. This avoided the hanging second model call while still testing schema failure and safe termination. The final adversarial experiment completed all 5 runs and reached the ceiling in 5/5 runs.

GitHub Submission Evidence



Final Summary:

In HW2, I extended my HW1 Clinical Trial Listings application in three main directions.

First, I improved the frontend so it remains usable at 375 px and added clear loading, empty, and error states. Second, I added a FastAPI backend that supports create, update, delete, and search operations. Finally, I refactored the earlier sequential Planner/Reviewer workflow into a LangGraph-based stateful supervisor architecture with Pydantic validation and bounded correction loops.

The experiments showed that the normal fixed input was highly reliable, with 30/30 valid first-attempt outputs. Both ceiling configurations achieved 100% completion, while the adversarial input reached the configured safety ceiling in 5/5 runs. The final verification script also reported:

```
overall_passed = true
```

Overall, the homework demonstrates both functional web/API integration & a more robust agent workflow with explicit state, routing, validation, and safety controls.